

Start coding or [generate](#) with AI.

Chương trình này là một framework mô phỏng mạch logic số và tự động sinh bảng chân lý (truth table). Nó cho phép bạn xây dựng các mạch gồm nhiều cổng logic (AND, OR, XOR, NOT, ...), kết nối chúng với nhau, sau đó thử tất cả các tổ hợp đầu vào để xem đầu ra tương ứng. Ngoài ra, chương trình còn có thể đọc mạch từ file và phân tích tự động.

 Chức năng chính của chương trình

 Chương trình dùng để:

 Mô phỏng mạch logic số

 Tính toán đầu ra từ các cổng logic

 Sinh bảng chân lý (truth table) cho toàn bộ mạch

 Nạp mạch từ file để phân tích tự động

 Thành phần chính

1. Các cổng logic (Gate Classes)

Chương trình định nghĩa đầy đủ các cổng:

AND, OR, XOR

NAND, NOR, XNOR

NOT

SWITCH, SPLIT

IN (input), OUT (output)

 Mỗi cổng:

Nhận input (True/False)

Tính toán output

Gửi tín hiệu sang cổng tiếp theo

<https://github.com/adam-rumpf/logic-circuit-truth-table/tree/master/circuits>

2. Cơ chế truyền tín hiệu _send_out()

 Khi một cổng xử lý xong:

Nó gửi output sang cổng khác Cổng kia tự động xử lý tiếp

 Giống như dòng điện chạy qua mạch

3. Lớp TruthTable

Đây là phần quan trọng nhất.

 Nhiệm vụ:

Thử mọi tổ hợp input Ghi lại output tương ứng

Ví dụ:

IN1 IN2 → OUT 0 0 → 0 0 1 → 1 1 0 → 1 1 1 → 0 4. Hàm number_to_boolean

 Chuyển số → vector boolean

Ví dụ:

5 → 101 → [True, False, True] 5. Hàm circuit_load

 Đọc mạch từ file:

Tạo các cổng Kết nối chúng Sinh bảng chân lý  Quy trình hoạt động Bước 1: Tạo mạch logic TestAnd0 = AndGate([TestOut]) Bước 2: Kết nối các cổng TestSplit = SplitGate([TestAnd10, TestAnd11]) Bước 3: Tạo bảng chân lý TestTable = TruthTable(...) Bước 4: Sinh bảng TestTable.generate_table()

 Chương trình sẽ:

Thử tất cả input In ra kết quả  Ý nghĩa thực tế

 Code này giống một:

 Simulator mạch số  Công cụ học: Digital logic Boolean algebra Computer architecture  Điểm cần chú ý  Nếu mạch có vòng lặp → có thể bị infinite loop  Cổng phải tạo theo thứ tự ngược (output → input)  Điểm mạnh  Linh hoạt (tự build mạch)  Hỗ trợ

nhiều loại cổng ✓ Có thể load từ file ✗ Hạn chế Không có GUI (chỉ in text) Không kiểm tra vòng lặp Không tối ưu cho mạch lớn 🔄 Tóm lại

_Example_2.txt

Name / Type / OutGates[] / OutPorts[]

Out OUT

And1 AND Out 0

And21 AND And1 0

And22 AND And1 1

Split SPLIT And21 And22 1 0

Xor31 XOR And21 0

Xor32 XOR Split 0

And33 AND And22 1

Switch1 SWITCH Xor31 Xor32 1 0

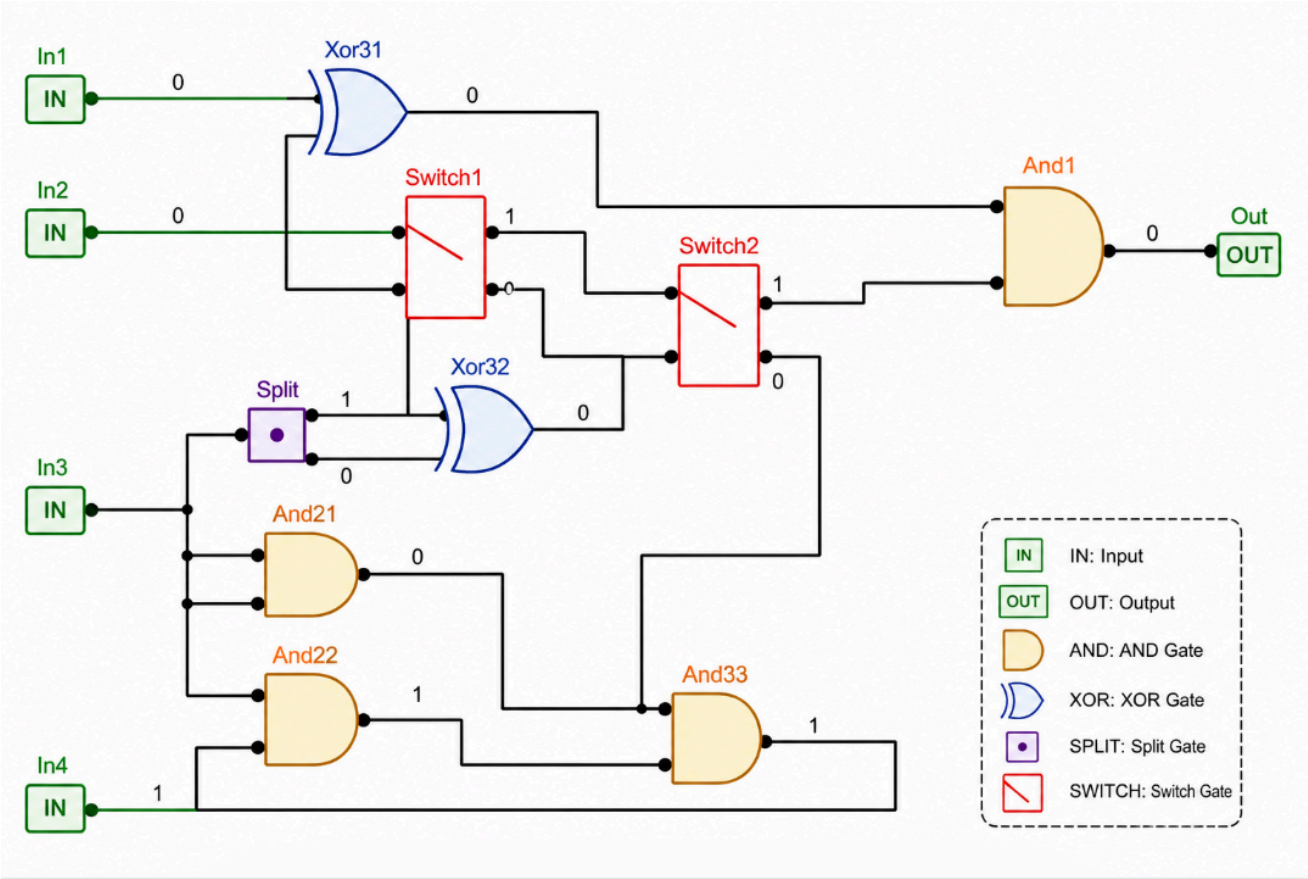
Switch2 SWITCH Xor32 And33 1 0

In1 IN Xor31 0

In2 IN Switch1 0

In3 IN Switch2 0

In4 IN And33 1



"""A module for generating truth tables of arrangements of logic gates.

This module includes a variety of classes to define logic gates and other related components. A logic circuit can be built by instantiating gate objects linked to each other. Most logic gates require one or more output gates, meaning that the circuit must generally be built beginning with the outputs and working backwards.

Most gates are instantiated in the same way, and require defining a list of output gates (one for each output). Gates which accept multiple inputs maintain

these inputs in a list. The keyword 'out_ports' argument accepts a list of port indices corresponding to the output gates. The keyword 'name' argument accepts a name for the gate to use in truth tables.

For example, if we already have two XOR gate objects called Xor1 and Xor2, then instantiating a SPLIT gate with SplitGate([Xor1, Xor2], out_ports=[1, 0]) would connect the SPLIT gate to port 1 of Xor1 and port 0 of Xor2. Note that even gates with a single output must be given their output gates in list form.

Most gates also include a process() method which causes it to evaluate its boolean inputs and calculate the appropriate boolean outputs, and a _send_out() method which sends its outputs to all output gates and prompts them to also evaluate their process() methods. This may lead to infinite loops if there are cycles in the logic circuit.

The IN and SWITCH gates both include a toggle() method which can be used to set their internal state. For IN, this means setting it to True or False, and for SWITCH, this means either transferring its output to its first ('left') or its second ('right') output gate.

The OUT gate has no output, and is mostly meant for use in defining truth table entries.

The TruthTable class can store sets of variable inputs, constant inputs, and outputs, and cycle through every possible setting of the variable inputs to generate a complete truth table of the resulting outputs via the generate_table() method.

The circuit_load() function can be used to load a circuit defined in an external text file in the /circuits/ folder. The format for a circuit file is described in the included _template file. As the input file is read, gate objects are created and linked to each other, and then used to instantiate a TruthTable object. The function returns a dictionary of the created gate objects as well as the created TruthTable object.

```

"""

import math

#####
# Global Functions
#####

=====
def number_to_boolean(num, bound):
    """Converts a number to a boolean representation of the binary expansion.

    Converts a decimal value to binary, and then the binary digits to a boolean
    vector. For example, number_to_boolean(5) would return [True, False, True]
    since 5 in binary is 101, and [1, 0, 1] in boolean is [True, False, True].
    """

    fstring = "{0:0}" + str(math.ceil(math.log(bound, 2))) + "b"

    return [bool(int(c)) for c in fstring.format(num)]

#####
# Gate Classes
#####

=====
class _Gate:
    """Generic gate template for use as a parent class."""

    #-----
    def __init__(self, ports, out_gates, out_ports=[0], name="<gate>"):
        """Constructor links gate to a given output gate.

        Requires a number of input ports and a list of output gate pointers.
        Optionally can specify port IDs of output gates and a name.
        """

        self.out_gates = out_gates # list of output gates
        self.out_ports = out_ports # indices of output gate ports
        self.name = name # name for use in string output
        self.inputs = [False for i in range(ports)] # list of input values
        self.outputs = [False for i in range(len(out_gates))] # output values

    #-----
    def __str__(self):
        """Default string is simply name."""

        return self.name

```

```

#-----
def process(self):
    """Placeholder input processor, overwritten by specific gates."""

    pass

#-----
def _send_out(self):
    """Sends internal state to output gate and prompts it to process."""

    for i in range(len(self.outputs)):
        self.out_gates[i].inputs[self.out_ports[i]] = self.outputs[i]
        self.out_gates[i].process()

#-----
def state_name(self):
    """String to represent internal state in truth tables."""

    if self.inputs[0] == False:
        return "F"
    elif self.inputs[0] == True:
        return "T"
    else:
        return "?"

#=====
class OutGate(_Gate):
    """OUTput which simply stores its input value."""

    #-----
    def __init__(self, name="<Out>"):
        """Constructor initializes several internal variables."""

        self.name = name
        self.inputs = [False]
        self.outputs = [False]

    #-----
    def process(self):
        """Input processor simply sets state to only input."""

        self.outputs[0] = self.inputs[0]

    #-----
    def _send_out(self):
        """Placeholder output sender to overwrite standard gate."""

        pass

#=====
class InGate(_Gate):
    """INput which simply outputs its input value."""

    #-----
    def __init__(self, out_gates, out_ports=[0], name="<In>"):
        """Constructor sets number of ports."""

        _Gate.__init__(self, 1, out_gates, out_ports=out_ports, name=name)

    #-----
    def process(self):
        """IN simply sends its own state to its output gate."""

        self._send_out()

    #-----
    def toggle(self, state):
        """Sets own internal state and sends to output."""

        self.inputs[0] = state
        self.outputs[0] = state
        self._send_out()

#=====
class TrueGate(_Gate):
    """Outputs a constant True signal."""

    #-----
    def __init__(self, out_gates, out_ports=[0], name="<True>"):
        """Constructor sets number of ports and sets input/output to True."""

        _Gate.__init__(self, 1, out_gates, out_ports=out_ports, name=name)

```

```

        self.inputs[0] = True
        self.outputs[0] = True

#-----
def process(self):
    """IN simply sends its own state to its output gate."""

    self._send_out()

#=====
class FalseGate(_Gate):
    """Outputs a constant False signal."""

#-----
def __init__(self, out_gates, out_ports=[0], name="<False>"):
    """Constructor sets number of ports and sets input/output to True."""

    _Gate.__init__(self, 1, out_gates, out_ports=out_ports, name=name)

    self.inputs[0] = False
    self.outputs[0] = False

#-----
def process(self):
    """IN simply sends its own state to its output gate."""

    self._send_out()

#=====
class AndGate(_Gate):
    """AND gate. Outputs True if both inputs are True."""

#-----
def __init__(self, out_gates, out_ports=[0], name="<And>"):
    """Constructor sets number of ports."""

    _Gate.__init__(self, 2, out_gates, out_ports=out_ports, name=name)

#-----
def process(self):
    """Evaluates own boolean inputs and sends result to output gate."""

    self.outputs[0] = self.inputs[0] and self.inputs[1]
    self._send_out()

#=====
class OrGate(_Gate):
    """OR gate. Outputs True if at least one input is True."""

#-----
def __init__(self, out_gates, out_ports=[0], name="<Or>"):
    """Constructor sets number of ports."""

    _Gate.__init__(self, 2, out_gates, out_ports=out_ports, name=name)

#-----
def process(self):
    """Evaluates own boolean inputs and sends result to output gate."""

    self.outputs[0] = self.inputs[0] or self.inputs[1]
    self._send_out()

#=====
class XorGate(_Gate):
    """XOR gate. Outputs True if exactly one input is True."""

#-----
def __init__(self, out_gates, out_ports=[0], name="<Xor>"):
    """Constructor sets number of ports."""

    _Gate.__init__(self, 2, out_gates, out_ports=out_ports, name=name)

#-----
def process(self):
    """Evaluates own boolean inputs and sends result to output gate."""

    self.outputs[0] = ((self.inputs[0] or self.inputs[1]) and not
                       (self.inputs[0] and self.inputs[1]))
    self._send_out()

#=====
class NandGate(_Gate):
    """NAND gate. Outputs True if at least one input is False."""

```

```

"""NAND gate. Outputs True if both inputs are False."""

#-----
def __init__(self, out_gates, out_ports=[0], name="<Nand>"):
    """Constructor sets number of ports."""

    _Gate.__init__(self, 2, out_gates, out_ports=out_ports, name=name)

#-----
def process(self):
    """Evaluates own boolean inputs and sends result to output gate."""

    self.outputs[0] = not (self.inputs[0] and self.inputs[1])
    self._send_out()

#=====
class NorGate(_Gate):
    """NOR gate. Outputs True if at least one input is False."""

#-----
def __init__(self, out_gates, out_ports=[0], name="<Nor>"):
    """Constructor sets number of ports."""

    _Gate.__init__(self, 2, out_gates, out_ports=out_ports, name=name)

#-----
def process(self):
    """Evaluates own boolean inputs and sends result to output gate."""

    self.outputs[0] = not (self.inputs[0] or self.inputs[1])
    self._send_out()

#=====
class XnorGate(_Gate):
    """XNOR gate. Outputs True if both inputs are the same."""

#-----
def __init__(self, out_gates, out_ports=[0], name="<Xnor>"):
    """Constructor sets number of ports."""

    _Gate.__init__(self, 2, out_gates, out_ports=out_ports, name=name)

#-----
def process(self):
    """Evaluates own boolean inputs and sends result to output gate."""

    self.outputs[0] = not (((self.inputs[0] or self.inputs[1]) and not
                           (self.inputs[0] and self.inputs[1])))
    self._send_out()

#=====
class NotGate(_Gate):
    """NOT Gate. Negates input signal."""

#-----
def __init__(self, out_gates, out_ports=[0], name="<Not>"):
    """Constructor sets number of ports."""

    _Gate.__init__(self, 1, out_gates, out_ports=out_ports, name=name)

#-----
def process(self):
    """Evaluates own boolean inputs and sends result to output gate."""

    self.outputs[0] = not self.inputs[0]
    self._send_out()

#=====
class DiodeGate(_Gate):
    """Diode. Simply carries signal unchanged, but only in one direction."""

#-----
def __init__(self, out_gates, out_ports=[0], name="<Diode>"):
    """Constructor sets number of ports."""

    _Gate.__init__(self, 1, out_gates, out_ports=out_ports, name=name)

#-----
def process(self):
    """Evaluates own boolean inputs and sends result to output gate."""

    self.outputs[0] = self.inputs[0]
    self._send_out()

```

```

=====
class SplitGate(_Gate):
    """Splits a signal into two outputs."""

    #-----
    def __init__(self, out_gates, out_ports=[0, 0], name="<Split>"):
        """Constructor sets number of ports."""

        _Gate.__init__(self, 1, out_gates, out_ports=out_ports, name=name)

    #-----
    def process(self):
        """Evaluates own boolean inputs and sends result to output gate."""

        self.outputs[0] = self.inputs[0]
        self.outputs[1] = self.inputs[0]
        self._send_out()

=====
class SwitchGate(_Gate):
    """Splitter that toggles between which output gate to trigger."""

    #-----
    def __init__(self, out_gates, out_ports=[0, 0], name="<Switch>"):
        """Constructor sets number of ports and initializes direction."""

        _Gate.__init__(self, 1, out_gates, out_ports=out_ports, name=name)

        self.direction = False # False for first output, True for second output

    #-----
    def process(self):
        """Evaluates own boolean inputs and sends result to output gate."""

        self.outputs[int(self.direction)] = self.inputs[0] # selected gate
        self.outputs[(int(self.direction)+1)%2] = False # non-selected gate
        self._send_out()

    #-----
    def toggle(self, state):
        """Sets direction and sends updated outputs."""

        self.direction = state
        self.process()

    #-----
    def state_name(self):
        """State name corresponds to direction of switch."""

        if self.direction == False:
            return "L"
        elif self.direction == True:
            return "R"
        else:
            return "?"

#####
# Truth Table Generation Class
#####

=====
class TruthTable:
    """Class for generating truth tables for a given logic circuit.

    Used to store a collection of logic gate inputs and outputs. Automatically
    checks every possible combination of inputs and displays the resulting
    outputs.
    """

    #-----
    def __init__(self, inputs=[], constants=[], outputs=[]):
        """Empty TruthTable class constructor initializes lists.

        Accepts optional keyword arguments to set the sets of input, constant,
        and output gates.
        """

        self.inputs = inputs # list of toggleable input gates
        self.constants = constants # list of constant input gates
        self.outputs = outputs # list of output gates

    #-----

```

```

def _get_outputs(self, input_values):
    """Returns all outputs for a given list of input values."""

    # Update each input and prompt all constant inputs to re-send signals
    for i in range(len(input_values)):
        self.inputs[i].toggle(input_values[i])
        for j in range(len(self.constants)):
            self.constants[j].process()

    # Return list of outputs
    return [out.state_name() for out in self.outputs]

#-----
def generate_table(self):
    """Generates outputs for every possible input combination."""

    # Check to see that the input/output sets are nonempty
    if len(self.inputs) == 0:
        print("No input gates defined. Quitting.")
        return None
    if len(self.outputs) == 0:
        print("No output gates defined. Quitting.")
        return None

    # Print label row
    row = ""
    for g in self.inputs:
        row += g.name + "\t"
    for g in self.outputs:
        row += g.name + "\t"
    print(row)

    for i in range(2**len(self.inputs)):
        # Use binary string to generate boolean vector
        input_set = number_to_boolean(i, 2**len(self.inputs))

        # Generate outputs from inputs
        output_set = self._get_outputs(input_set)

        # Print values
        row = ""
        for g in self.inputs:
            row += g.state_name() + "\t"
        for j in output_set:
            row += str(j) + "\t"
        print(row)

#####
# Circuit File Reading Function
#####

#=====
def circuit_load(file_name):
    """Loads a circuit file from the /circuits/ folder into a TruthTable.

    Requires the name of a circuit file. Outputs (as a tuple) a dictionary of
    the created gate objects (indexed by name) and pointer to the generated
    TruthTable object.

    The specified file name is assumed to be in the /circuits/ folder. See the
    included file '_template.txt' for an explanation of the file format, and
    '_example01.txt' and '_example02.txt' for an example circuit file.
    """

    # Initialize dictionary associating gate names with gate objects
    gates = {}

    # Initialize TruthTable lists
    inputs = []
    constants = []
    outputs = []

    # Open file and read line-by-line
    with open("/content/sample_data/circuits/"+file_name, 'r') as f:
        for line in f:

            # Skip comment or blank lines
            if (line[0] == "#") or (len(line) <= 1):
                continue

            # Get gate name and type
            line_split = line.split()

```



```

gate_name = line_split[0]
gate_type = line_split[1].upper()

# Main try/except block to catch key errors and index errors
try:

    # Break into gate type category
    if gate_type in {"OUT"}:
        # Outputs

        # Check number of arguments
        if len(line_split) != 2:
            raise IndexError

        if gate_type == "OUT":
            gates[gate_name] = OutGate(name=gate_name)
            outputs.append(gates[gate_name])

    elif gate_type in {"IN", "TRUE", "FALSE"}:
        # Inputs

        # Check number of arguments
        if len(line_split) != 4:
            raise IndexError

        # All inputs specify one output
        out = [gates[line_split[2]]]
        port = [int(line_split[3])]

        # Create the specified type of input
        if gate_type == "IN":
            gates[gate_name] = InGate(out, out_ports=port,
                                      name=gate_name)
            inputs.append(gates[gate_name])
        elif gate_type == "TRUE":
            gates[gate_name] = TrueGate(out, out_ports=port,
                                       name=gate_name)
            constants.append(gates[gate_name])
        elif gate_type == "FALSE":
            gates[gate_name] = FalseGate(out, out_ports=port,
                                         name=gate_name)
            constants.append(gates[gate_name])

    elif gate_type in {"AND", "OR", "XOR", "NAND", "NOR", "XNOR"}:
        # Binary

        # Check number of arguments
        if len(line_split) != 4:
            raise IndexError

        # All binary gates specify one output
        out = [gates[line_split[2]]]
        port = [int(line_split[3])]

        # Create the specified type of binary gate
        if gate_type == "AND":
            gates[gate_name] = AndGate(out, out_ports=port,
                                       name=gate_name)
        elif gate_type == "OR":
            gates[gate_name] = OrGate(out, out_ports=port,
                                      name=gate_name)
        elif gate_type == "XOR":
            gates[gate_name] = XorGate(out, out_ports=port,
                                       name=gate_name)
        elif gate_type == "NAND":
            gates[gate_name] = NandGate(out, out_ports=port,
                                        name=gate_name)
        elif gate_type == "NOR":
            gates[gate_name] = NorGate(out, out_ports=port,
                                       name=gate_name)
        elif gate_type == "XNOR":
            gates[gate_name] = XnorGate(out, out_ports=port,
                                        name=gate_name)

    elif gate_type in {"NOT", "DIODE"}:
        # Unary

        # Check number of arguments
        if len(line_split) != 4:
            raise IndexError

        # All unary gates specify one output
        out = [gates[line_split[2]]]

```

```

port = [int(line_split[3])]

# Create the specified type of unary gate
if gate_type == "NOT":
    gates[gate_name] = NotGate(out, out_ports=port,
                               name=gate_name)
elif gate_type == "DIODE":
    gates[gate_name] = DiodeGate(out, out_ports=port,
                                  name=gate_name)

elif gate_type in {"SPLIT", "SWITCH"}:
    # Splitter

    # Check number of arguments
    if len(line_split) != 6:
        raise IndexError

    # All splitter gates specify two outputs
    out = [gates[line_split[2]], gates[line_split[3]]]
    port = [int(line_split[4]), int(line_split[5])]

    # Create the specified type of splitter
    if gate_type == "SPLIT":
        gates[gate_name] = SplitGate(out, out_ports=port,
                                      name=gate_name)
    elif gate_type == "SWITCH":
        gates[gate_name] = SwitchGate(out, out_ports=port,
                                       name=gate_name)
        inputs.append(gates[gate_name])

else:
    # Unknown
    print("Error: Unknown gate type '"+gate_type+"' read.")
    print("Quitting.")
    return {}, TruthTable()

except KeyError as e:
    # A key error occurs if the circuit file mentions an output
    # gate that has not yet been defined
    print("Error: Gate '"+gate_name+"' lists output gate '"+
          e.args[0]+' which has not yet been defined.")
    print("Gates must be defined in reverse topological order.")
    print("Quitting.")
    return {}, TruthTable()

except IndexError:
    # An index error occurs if too few parameters are listed for
    # gates with multiple outputs
    print("Error: Incorrect number of arguments listed for gate '"+
          gate_name+"'.")
    print("Quitting.")
    return {}, TruthTable()

# Create TruthTable object
table = TruthTable(inputs=inputs, constants=constants, outputs=outputs)

print("Circuit file successfully read.")

return gates, table

#####
# Execution
#####

# Example
if __name__ == "__main__":

    # Define a system of logic gates
    TestOut = OutGate(name="OUT")
    TestAnd0 = AndGate([TestOut])
    TestAnd10 = AndGate([TestAnd0], out_ports=[0])
    TestAnd11 = AndGate([TestAnd0], out_ports=[1])
    TestSplit = SplitGate([TestAnd10, TestAnd11], out_ports=[1, 0])
    TestXor20 = XorGate([TestAnd10], out_ports=[0])
    TestXor21 = XorGate([TestSplit])
    TestAnd22 = AndGate([TestAnd11], out_ports=[1])
    TestSwitch0 = SwitchGate([TestXor20, TestXor21], out_ports=[1, 0],
                             name="Switch1")
    TestSwitch1 = SwitchGate([TestXor21, TestAnd22], out_ports=[1, 0],
                             name="Switch2")
    TestIn0 = InGate([TestXor20], out_ports=[0], name="IN1")
    TestIn1 = InGate([TestSwitch0], name="IN2")
    TestIn2 = InGate([TestSwitch1], name="IN3")

```

```

TestIn3 = InGate([TestAnd22], out_ports=[1], name="IN4")

# Define a truth table object
TestTable = TruthTable(inputs=[TestIn0, TestIn1, TestIn2, TestIn3,
                               TestSwitch0, TestSwitch1],
                       outputs=[TestOut])

# Generate and display truth table
# TestTable.generate_table()

# Load a circuit from a file and generate its truth table
print("_example02.txt \n")
_, LoadTable = circuit_load("_example02.txt")
print("\n")
LoadTable.generate_table()

```

_example02.txt

Circuit file successfully read.

Switch1	Switch2	In1	In2	In3	In4	Out
L	L	F	F	F	F	F
L	L	F	F	F	T	F
L	L	F	F	T	F	F
L	L	F	F	T	T	F
L	L	F	T	F	F	F
L	L	F	T	F	T	F
L	L	F	T	T	F	F
L	L	F	T	T	T	F
L	L	T	F	F	F	F
L	L	T	F	F	T	F
L	L	T	F	T	F	F
L	L	T	F	T	T	F
L	L	T	T	F	F	F
L	L	T	T	F	T	F
L	L	T	T	T	F	F
L	L	T	T	T	T	F
L	R	F	F	F	F	F
L	R	F	F	F	T	F
L	R	F	F	T	T	F
L	R	F	T	F	F	F
L	R	F	T	F	T	F
L	R	F	T	T	F	F
L	R	F	T	T	T	F
L	R	F	T	T	T	F
L	R	T	F	F	F	F
L	R	T	F	F	T	F
L	R	T	F	T	F	F
L	R	T	F	T	T	F
L	R	T	T	F	F	F
L	R	T	T	F	T	F
L	R	T	T	T	F	F
L	R	T	T	T	T	F
R	L	F	F	F	F	F
R	L	F	F	F	T	F
R	L	F	F	T	F	F
R	L	F	T	F	F	F
R	L	F	T	F	T	F
R	L	F	T	T	F	F
R	L	F	T	T	T	F
R	L	F	T	T	T	F
R	L	T	F	F	F	F
R	L	T	F	F	T	F
R	L	T	F	T	F	F
R	L	T	F	T	T	F
R	L	T	T	F	F	F
R	L	T	T	F	T	F
R	L	T	T	T	F	F
R	L	T	T	T	T	F
R	R	F	F	F	F	F
R	R	F	F	F	T	F
R	R	F	F	T	F	F
R	R	F	F	T	T	F

